

Proyecto 2 - Diseño para la Gestión de Inventario en Tienda

Diego Andre Calderón Salazar - 241263

Universidad del Valle de Guatemala

Base de Datos 1

Facultad de Ingeniería - Ciencia de la Computación y Tecnologías de la información

Contexto del Negocio	2
Stack Seleccionado	3
Base de Datos	3
pendiente	3
Diseño de la Base de Datos y Normalización	3
Entidades Identificadas inicialmente	3
Dependencias funcionales iniciales	5
Normalización	6
Dependencias funcionales actualizadas	9
Relaciones y Cardinalidades	10
Diagrama Entidad Relación (DER)	10
Modelo Relacional	10
Formato texto	10
Diagrama	11

Contexto del Negocio

La tienda maneja productos agrupados en categorías, comprados a proveedores. Los clientes realizan compras atendidas por empleados. Cada compra puede incluir varios productos y debe quedar registrada junto con el detalle de lo vendido. La tienda necesita controlar el stock disponible y generar reportes de venta.

Stack Seleccionado

Base de Datos

Se optó por MariaDB por su eficiencia en entornos de baja escala y su facilidad para gestionar credenciales específicas vía variables de entorno. Dado que el sistema no exige una robustez empresarial, este motor permite un desarrollo ágil de las funcionalidades CRUD y consultas SQL explícitas requeridas.

pendiente

Diseño de la Base de Datos y Normalización

Entidades Identificadas inicialmente

- Producto
 - **Id**
 - Nombre
 - Precio
 - Id_categoría
 - Id_proveedor
- Categoría
 - **Id**
 - Detalle
- Proveedor

- **Id**
- Nombre
- Cliente (se puede sacar luego algunos atributos a una entidad persona)
 - **Id**
 - Nombre
 - Nit
 - Correo (opcional)
 - Direccion (opcional)
- Empleado
 - **Id**
 - Nombre
 - DPI
- Compra (un empleado hace muchas ventas y un cliente hace muchas compras)
(una compra tiene muchos productos y un producto tiene muchas compras)
 - **Id**
 - Id_cliente
 - Id_empleado
 - Fecha (DATETIME para que guarde tambien la hora)
- Detalle-venta (tabla de cruce)
 - **Id compra**
 - **Id producto**
 - Cantidad
 - Subtotal (precio * cantidad)
- Stock
 - Id

- Id_producto
- Cantidad

No hice entidad de reportes ya que prefiero realizarlos mediante consultas de manera dinámica en la web y luego mostrarlos o dar la opción de imprimirlo.

Dependencias funcionales iniciales

- Producto
 - Id -> Nombre, id_proveedor, id_categoria, precio
- Categoría
 - Id -> Detalle
- Proveedor
 - Id -> nombre
- Cliente
 - Id -> Nombre, Correo, nit, dirección
 - NIT no implica nombre ni dirección debido a que el NIT puede ser CF y la dirección registrada en SAT al NIT puede ser distinta a la dirección de entrega.
- Empleado
 - Id -> Nombre, DPI
- Compra
 - Id -> id_cliente, id_empleado, fecha
- Compra-producto
 - (id_compra, id_producto) -> cantidad, subtotal
 - Id_producto -> subtotal
 - Cantidad -> subtotal

- Rompe 2NF, pues existe una dependencia parcial donde subtotal depende únicamente del id_producto (por el cálculo de precio * cantidad)
- Stock
 - Id -> id_producto, cantidad

Normalización

- Producto
 - **Id**
 - Nombre
 - Precio
 - Id_categoría (FK)
 - Id_proveedor (FK)
 - Stock
 - Como la relación entre la entidad stock y producto es 1:1, decidí agregar un atributo llamado stock a la entidad de producto, que registrará la cantidad de productos restantes existentes. Por este motivo eliminé la entidad Stock.
- Categoría
 - **Id**
 - Detalle
 - Esta entidad se hizo para ahorrar memoria en la base de datos y no repetir en varias tuplas los mismos strings.
- Proveedor
 - **Id**
 - Nombre

- Esta entidad se hizo para ahorrar memoria en la base de datos y no repetir en varias tuplas los mismos strings.
- Persona
 - **Id**
 - Nombre
 - Correo
 - Contraseña
 - Id_rol (FK)
 - Agregué la entidad persona para poder tener una entidad que guardé los datos personales tanto de los empleados como de los clientes, y también para almacenar información que va a ser útil para la funcionalidad del login (como la contraseña, el correo y el rol)
- Rol
 - **Id**
 - Detalle (admin, vendedor, cliente)
 - Esta entidad se hizo para ahorrar memoria en la base de datos y no repetir en varias tuplas los mismos strings.
- Cliente
 - **Id**
 - Id_persona (FK)
 - NIT
 - Direccion (opcional)
- Empleado
 - **Id**

- Id_persona (FK)
- DPI
- Compra
 - **Id**
 - Id_cliente (FK)
 - Id_empleado (FK)
 - Fecha (DATETIME para que guarde tambien la hora)
- Detalle-venta (tabla de cruce)
 - **Id compra** (FK)
 - **Id producto** (FK)
 - Cantidad
 - Precio_unitario
 - Se guarda acá además de en producto ya que el precio en ciertos momentos puede cambiar, y es importante guardar el valor del precio en ese preciso momento para luego calcular el subtotal (cantidad * precio_unitario)
 - Esto resuelve la inconsistencia que rompía la normalización de 2NF ya que ahora Precio unitario depende tanto de id_compra como de id_producto, pues es un precio estático que se extrae del producto en la fecha de realizada cierta compra. En este caso tampoco existe la dependencia de Cantidad -> subtotal debido a que ahora el subtotal se calcula fuera utilizando el precio unitario y la cantidad.

Dependencias funcionales actualizadas

- Producto
 - Id -> Nombre, id_proveedor, id_categoria, precio
- Categoría
 - Id -> nombre
- Proveedor
 - Id -> nombre
- Persona
 - Id -> Nombre, correo, contraseña, id_rol
- Rol
 - Id -> detalle
- Cliente
 - Id -> id_persona, NIT, Direccion
- Empleado
 - Id -> id_persona, DPI
- Compra
 - Id -> id_cliente, id_empleado, fecha
- Detalle-venta
 - (id_compra, id_producto) -> Cantidad, precio_unitario

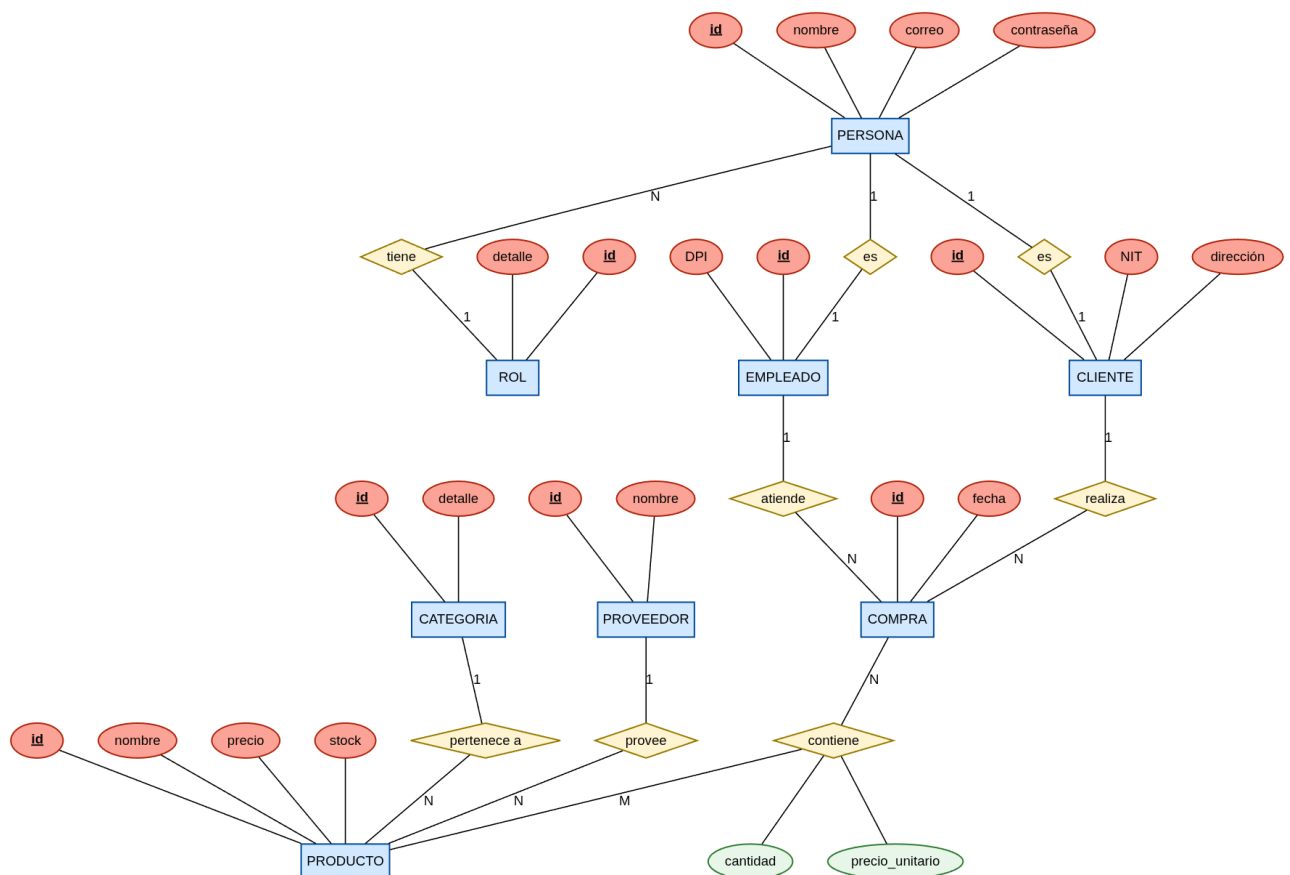
El diseño de la base de datos se encuentra normalizado en 3FN. Se eliminaron las dependencias parciales al asegurar que el precio_unitario sea un registro histórico dependiente de la llave compuesta en la tabla de cruce. Asimismo, se descartaron las dependencias transitivas al separar los datos personales en la entidad Persona y tratar el

subtotal como un atributo derivado calculado en la capa de aplicación. Esta arquitectura garantiza la integridad referencial y optimiza el rendimiento mediante una segmentación lógica de entidades.

Relaciones y Cardinalidades

Entidad 1	Relación	Entidad 2	Cardinalidad
Producto	Pertenece a	Categoría	N:1
Proveedor	Provee	Producto	1:N
Cliente	Realiza	Compra	1:N
Empleado	Atiende	Compra	1:N
Empleado	Es	Persona	1:1
Cliente	Es	Persona	1:1
Persona	tiene	Rol	N:1
Compra	tiene	Producto	N:M

Diagrama Entidad Relación (DER)



Modelo Relacional

Formato texto

- **ROL** (**ID**, detalle)
- **PERSONA** (**ID**, nombre, correo, contraseña, **id_rol** → **ROL**)
- **CLIENTE** (**ID**, **id_persona** → **PERSONA**, NIT, direccion)
- **EMPLEADO** (**ID**, **id_persona** → **PERSONA**, DPI)
- **CATEGORIA** (**ID**, detalle)
- **PROVEEDOR** (**ID**, nombre)
- **PRODUCTO** (**ID**, nombre, precio, stock, **id_categoria** → **CATEGORIA**,
id_proveedor → **PROVEEDOR**)
- **COMPRA** (**ID**, fecha, **id_cliente** → **CLIENTE**, **id_empleado** →
EMPLEADO)
- **DETALLE_VENTA** (**id_compra** → **COMPRA**, **id_producto** →
PRODUCTO, cantidad, precio_unitario)

Diagrama

